

RG660MK 遥操时延优化 MVP 模拟部署报告

飞书 / Hermes 控制链路 + QoS 带宽分配 + 一拖多能力验证

总体结论：QoS 机制已验证有效，但核心业务目标尚未证明。

在 br-lan 出口上，QoS 使本地排队时延 P95 下降 **92.4%**、反馈流抖动 P95 下降 **86.3%**，并把视频流从「被背景流量挤到几乎饿死」中救回（1.28 → 4.57 Mbit/s）。

但 **CONTROL（飞书控制链路）不走 br-lan 而走 ccmni2**，本次未对该接口下发 任何调度规则 —— 因此控制时延的改善**无法用一个共同瓶颈证明**，两项 CONTROL 判据未达标。这是本 PoC 最重要的遗留项。

目标设备：RG660MK（OpenWrt 23.05.0 r23497 / kernel 5.15.134 / aarch64 Cortex-A55）

测试形态：MVP 模拟部署（非生产割接），运行时 **tc**，无持久化

依据文档：《RG660MK 遥操时延优化 MVP 模拟部署设计文档 V1.1》+ 补充说明 § 一～§ 十四

报告日期：2026-09-23

数据来源：现场实测，全部原始数据随报告归档（见附录 A）

目录

- 一 设计思路
- 二 测试结果
- 三 解决了什么问题
- 四 遗留了什么
- 五 预计如何解决
- 附 附录 A 交付物清单 / 附录 B 复现步骤 / 附录 C 安全约束遵守情况

一、设计思路

1.1 要回答的问题

远端遥操作场景下，一台 RG660MK 同时承载三类流量：**控制指令**（飞书 → Hermes）、**实时视频**（camview MJPEG）、**音频反馈**（麦克风回传）。链路带宽有限时，三者会相互挤占。本 PoC 要回答的是：

- 在**拥塞**发生时，给出口加一套优先级调度（QoS），能否把控制与反馈的时延、抖动、丢包保住？
- 这套调度会不会在**无拥塞**时反而拖慢正常业务？
- 在 6 Mbit/s 级瓶颈下，**最多能同时支撑几路逻辑机器人组**？

方法论前提：没有拥塞，QoS 就没有可调度的东西

本 PoC 不比较「不同负载下的两个结果」。所有对比都在**相同人工瓶颈 + 相同业务负载**下进行，唯一变量是调度方式。无拥塞时的 ON/OFF 差异小是**预期现象**，不是 QoS 无效的证据。

1.2 硬约束（贯穿全过程的红线）

不刷机 · 不重启设备 · 不破坏现有摄像头服务 · 不永久修改网络拓扑 · 不覆盖未知的既有 QoS · 不修改 camview 持久配置 · 不泄露任何 token/secret/cookie · 不把所有 TCP/443 无差别标记成最高优先级 · 测试结束必须清理 tc 规则与临时进程 · 发生 SSH 不稳定 / OOM / camview 异常立即中止并回滚。

上述约束直接决定了技术选型：QoS 只能用**运行时 tc**（不写 `/etc/config`），所有配置必须可秒级撤销，且必须有一个「控制端失联也能自动拆除」的安全机制。

1.3 三种测试状态（A/B/C 对照）

状态	瓶颈	调度	作用
CASE-A IDLE	无	内核默认	真实底噪参考
IDLE-QoS	无	6M HTB 分类	回答「无拥塞时 QoS 是否影响业务」
CASE-B 拥塞 + QoS OFF	6 Mbit/s 单队列 TBF	无分类	对照组
CASE-C 拥塞 + QoS ON	6 Mbit/s HTB 分类	4 类优先级	实验组（负载与 B 完全一致）

每个状态至少 3 轮，取轮间分布判断效应是否大于离散。

1.4 QoS 设计

在设备出口（egress）挂一棵 HTB 树，把出口切成 4 个优先级类别。关键设计取舍：

Class	业务	HTB prio	保底 rate	上限 ceil
1:10	CONTROL（飞书/Hermes）	0	512 kbit/s	6 Mbit/s
1:15	MGMT（SSH / ICMP）	0	512 kbit/s	6 Mbit/s
1:20	FEEDBACK（音频回传）	1	256 kbit/s	1 Mbit/s
1:30	VIDEO（camview）	2	4 Mbit/s	5.5 Mbit/s
1:40	BEST_EFFORT（default）	3	256 kbit/s	6 Mbit/s

保底之和 5536 kbit/s < 6000 kbit/s，层级预算成立。叶子队列统一 `fq_codel`。**MGMT 单独成类**是刻意设计：SSH 一旦被饿死，人就失去了干预手段，安全网随之失效。

分类规则（u32，取自 `tc` 实际 dump）

prio	匹配	classid
1	<code>match 00160000/ffff0000 at 20</code> （ip sport 22）	1:15 MGMT
2	<code>match 00001580/0000fff0 at 20</code> （udp dport 5504~5519）	1:20 FEEDBACK
3	<code>match 1f9a0000/ffff0000 at 20</code> （tcp sport 8090）	1:30 VIDEO
6	<code>match 00010000/00ff0000 at 8</code> （ip proto 1 / ICMP）	1:15 MGMT
—	未匹配 → HTB <code>default 0x40</code>	1:40 BEST_EFFORT

内核未编译 `flower`，按设计文档 § 9.3 回退到 `u32`。FEEDBACK 用端口段（5504 + 掩码 0xfff0）而非单端口，是为了支持一拖多——这个细节后来成为一个重要缺陷的根源（见 3.2）。

1.5 指标设计：三种时延严格区分

本 PoC 最容易出错的地方是把不同来源的时延混为一谈。因此从一开始就分成三个独立量具：

量具	测什么	为什么需要它
CONTROL-APP	飞书→Hermes→应答的端到端事务	唯一能代表「用户真实感受」的指标。 本次未采集 （见 4.2）
CONTROL-NET	真实 CONTROL 五元组上的应用层心跳 RTT（被动 pcap 提取）	排除 LLM 生成时间，只看网络
NET_PROBE	PC↔设备 ICMP / 设备↔飞书 TCP 握手	只用于解释本地队列 ，不能替代真实业务

波动指标统一定义为 $\text{variation}(i) = |x(i) - x(i-1)|$ （相邻样本差），并明确命名为 **应用事务时延波动**，不冒充纯网络 jitter。本 PoC **没有 PTP**，因此不做单向时延；jitter 只用接收端同一个 monotonic clock 计算。

1.6 安全机制

因为设备是远端且承载生产业务，`qos_apply.sh` 内置 **dead-man 看门狗**：下发 qdisc 的同时启动一个后台循环，每 10 s 检查一次 keepalive 文件；超过 90 s 未刷新，就**自动拆除 qdisc**。这样即使控制端崩溃、SSH 断开、人离开

现场，设备也不会停留在「被限速」的状态。

这个机制本身也踩了坑

采集窗口长度与 GRACE 同为 90 s，而早期版本只在窗口**开始前**喂一次 keepalive —— 看门狗于是在窗口末尾把 qdisc 拆了，测出来的是「没有 QoS 的裸链路」。属于典型的**静默失效**：脚本不报错、结果看起来正常、但全是错的。发现与修复过程见 3.2。

二、测试结果

2.1 关键结论



2.2 A/B 对比结果（CASE-B vs CASE-C，各 3 轮）

改善% 一律正数表示变好；吞吐类已按「越高越好」折算。

指标	CASE-B (OFF)	CASE-C (ON)	改善%	判定
NET_PROBE-LAN RTT P95（本地排队）	40.467 ms	3.064 ms	92.43%	达标
NET_PROBE-LAN RTT P99	42.908 ms	3.288 ms	92.34%	达标
NET_PROBE-LAN 时延波动 P95	3.233 ms	1.065 ms	67.06%	达标
FEEDBACK Jitter P95	3.197 ms	0.438 ms	86.30%	达标
FEEDBACK Jitter P99	4.340 ms	1.329 ms	69.38%	达标
FEEDBACK Jitter max	17.898 ms	8.757 ms	51.07%	达标
FEEDBACK 丢包率	0.0%	0.0%	—	达标
VIDEO 吞吐	1.284 Mbit/s	4.57 Mbit/s	+255.92%	达标
CONTROL-NET 心跳 RTT P95	79.348 ms	69.507 ms	12.40%	未达标
NET_PROBE-WAN RTT P95	86.729 ms	74.342 ms	14.28%	未达标

这些改善是真的：每一项达标的指标，OFF/ON 区间都不重叠

即「QoS 打开」带来的效应大于同一条件下的轮间离散。反之，两项 CONTROL 指标的 OFF/ON 区间**相互重叠**—— 那 12~14% 是公网抖动，不能算作 QoS 的功劳。

关于 VIDEO：不是「让出带宽」，而是「被救出来」

预期里 QoS ON 时 VIDEO 应当让带宽给控制与反馈（设计文档 S 六 明确允许其吞吐下降）。实测出现了相反的情况，且这不是测量错误 —— 用两条独立路径交叉验证过：

轮次	PC 侧实收	设备侧 1:30 类计数	br-lan 总出口
congested_qos_off ×3	1.279 / 1.284 / 1.288	—	5.433 / 5.439 / 5.440
congested_qos_on ×3	4.509 / 4.523 / 4.678	4.211 / 4.220 / 4.327	5.507 / 5.495 / 5.506

两个量具在 ON 轮次吻合到 8% 以内。**两个 arm 的总出口几乎相同**（5.43~5.44 vs 5.50~5.51 Mbit/s），说明瓶颈在两个 arm 里都被同样打满——差别不在「压了多少」，而在「怎么分」。QoS OFF 时 VIDEO 只拿到 1.28 Mbit/s，其余约 4.15 Mbit/s 被无差别的背景 TCP 吃掉；QoS ON 时它拿到 4.2~4.3 Mbit/s。

这条结论对正式部署很重要

在不做分类的出口上，**视频流是最容易被牺牲的那一个**（它是「弹性」流量，会让出带宽）。如果生产环境里存在任何背景大流量，视频质量会先于控制链路崩掉。

2.3 无拥塞时 QoS 的影响（CASE-A）

指标	IDLE	IDLE-QoS	差异	判读
VIDEO 吞吐	4.538	4.534	-0.09%	无影响
FEEDBACK Jitter P95	0.452	0.474	-4.87%	无影响
FEEDBACK 丢包率	0.0	0.0	—	无丢包
NET_PROBE-LAN RTT P95	2.504 ms	3.039 ms	-21.37%	固定开销

结论应表述为「**基本不影响，但不是零影响**」：吞吐与抖动不受影响；本地 ICMP 路径的 P95 付出约 **0.5 ms 的固定开销**——这是 HTB 分类与 fq_codel 入队出队的固有成本（`idle` 组用的是内核默认 `noqueue`，不做任何排队）。绝对值很小，但必须如实报告。

一个更有说服力的对照：拥塞 + QoS ON 时的本地 P95 是 3.064 ms，与无拥塞的 2.504 ms 基本相同——也就是说，**QoS 把「拥塞下的本地时延」拉回到了「不拥塞时」的水平**。

2.4 一拖多能力（最大 N）

一组 = 1 路 VIDEO + 1 路 FEEDBACK + 1 路 CONTROL。现场只有一台设备、一路摄像头、一路麦克风，N 组复用同一路真实源，因此该 N 反映的是**出口带宽与排队能撑住几组**，不是「能否同时编码 N 路摄像头」。

N	结论	FEEDBACK 丢包	Jitter P95	Jitter P99	相对 N=1	VIDEO 合计	CPU
1	PASS	0.0%	0.506 ms	1.496 ms	—	4.528 Mbit	15.92%
2	PASS	0.0%	1.817 ms	2.765 ms	3.59×	5.520 Mbit	20.25%
4	FAIL	0.0%	10.762 ms	14.265 ms	21.27×	5.410 Mbit	16.75%
8	FAIL	17.692%	69.917 ms	73.185 ms	138.18×	5.024 Mbit	24.03%
16	FAIL	59.291%	160.762 ms	170.056 ms	317.71×	5.024 Mbit	26.69%

最大能力 N = 2（定义：最后一个**同时**满足 CONTROL / FEEDBACK / 系统稳定性要求的 N，不是「还能发包」的最大值）。

瓶颈定位：为什么恰好断在 N=2 和 N=8 之间

实测每路 FEEDBACK 约占用 **134 kbit/s** (336 B × 50 pps)，则 N 路总需求 = 134 × N kbit/s。把这个需求与 1:20 类的两个参数对照，瓶颈的两道门槛就清楚了：

参数	值	作用	推算的容量上限	实测首次失效
rate 保底	256 kbit/s	决定排队时延（超了就在队列里等 → jitter 上升）	≈ 1.9 路	N=2 (jitter 超标)
ceil 上限	1 Mbit/s	决定丢包（超了 fq_codel 尾丢）	≈ 7 路	N=8 (开始丢包)

两个判据在不同的 N 上先后失效，且与类别计数吻合：1:20 类实测在 N=8/16 时稳定饱和到 0.918 / 0.915 Mbit/s —— 正是 1 Mbit/s 的硬上限。

关键区分：rate 管抖动，ceil 管丢包

N=4 时丢包为 0% 而 jitter 已超标（10.8 ms）—— 「包都到了，只是到达时刻被拉长」。若只看丢包率，会误判 N=4 通过。这是本 PoC 坚持同时考核两个维度的原因。

2.5 系统稳定性

检查项	结果
CPU 占用	QoS ON 平均 17.25%，峰值 26.69%，未长期超过 85% 阈值
内存 / OOM	未发生 OOM，未发生内核崩溃
camview / camav / camview6 / Hermes gateway	全程存活，未重启
SSH 可连接性	全程可连（MGMT 类保护生效）
网卡持续 TX drop / error	未出现
设备重启	未重启（uptime 持续增长）

2.6 设计文档 § 15.3 十一个必答问题

编号沿用设计文档原文（13–23）。

#	问题	回答
13	实际网口与链路速率	br-lan 成员 eth0 2500M / eth1 1000M，设备→PC 实测 937 Mbit/s ；CONTROL 出公网走 ccmni2（LTE）
14	哪些 tc 组件可用	htb / hfsc / fq_codel 内置； tbft 需 modprobe； u32 、 fw 可用； flower / cake / sfq 不可用
15	CONTROL 连接形态与隔离方法	TCP 10.128.237.160:36446 → 211.95.51.16:443 （open.feishu.cn），属 hermes gateway；LLM 走 140.210.138.13。按 目的 IP 精确匹配 标记，未用 tcp dport 443 通配
16	camview 真实媒体五元组	192.168.1.1:8090 → 192.168.1.244:<临时端口> TCP / MJPEG；按 sport 8090 分类
17	麦克风设备与采样格式	真实 C270， plughw:1 @ 8 kHz / 16 bit / 单声道 。注意并非补充说明假设的 16 kHz——这是设备侧实测能力，帧长取 20 ms / 320 B
18	LINK_RATE 取值与理由	6000 kbit/s 。拥塞窗口内 offered load 约 5.5 Mbit/s，取 6 Mbit/s 使拥塞 必然发生且可重复 ，同时给 CONTROL/FEEDBACK 留出可分配的保底空间
19	QoS OFF 时各项指标	见 2.2 表 CASE-B 列。CONTROL-APP（含 LLM 的真实事务） 未采集 ，见 4.2
20	QoS ON 改善% 与是否命中 P0 类	改善见 2.2。 控制连接没有命中 P0 类——这是实测结论 ：1:10 分类器已下发且规则核对无误，但所有 ON 轮次 cls_control_bytes 均为 0 。原因是 CONTROL 经 ccmni2 出网，而 HTB 只挂在 br-lan
21	FEEDBACK / VIDEO 吞吐、连续性、丢包	见 2.2 与 2.3。FEEDBACK 丢包与 jitter 显著改善；VIDEO 吞吐不降反升
22	一拖多最大 N 与首个失败 N 的瓶颈	最大 N=2 ；首个失败 N=4，瓶颈是 1:20 类 rate 保底 256 kbit/s 不足 （详见 2.4）
23	迁移到 QCS8550 的最大技术差距	无对照实测，只能给结构判断：A55 为小核，高包速率下 tc /softirq 开销余量更紧；8550 可用 flower 等高级分类。见 4.4

三、解决了什么问题

3.1 用真实数据证明了 QoS 机制有效（本地出口侧）

在相同瓶颈、相同负载下，QoS 使本地排队时延 P95 从 40.5 ms 降到 3.1 ms，且达到与无拥塞时相当的水平（2.5 ms）。反馈流抖动 P95 下降 86.3%，丢包保持 0%。这些结论都有 OFF/ON 区间不重叠作为支撑，不是单点采样。

同时确认了 QoS 的代价：无拥塞时本地 ICMP P95 增加约 0.5 ms 固定开销。这个数字很小但是真实存在的，正式部署做容量规划时应计入。

3.2 定位并修复了 6 个脚手架缺陷（其中 4 个属静默失效）

这部分的价值不低于测试数据本身。**静默失效**指脚本不报错、日志正常、结果看起来像一个合法的结论，但实际测的东西是错的——如果没被发现，报告会输出一个完全错误的能力结论。

#	缺陷	表现	如何发现的
1	FEEDBACK 分类漏配	u32 规则写成单端口 5502/0xffff，一拖多时只有第 0 路进 FEEDBACK 类，其余 N-1 路全部回落到 BEST_EFFORT。 后果： 把「脚手架漏配」误读成「设备撑不住 N 路」，直接污染最大能力结论	异常形态：feedback_0 在每个 N 都完美，而 feedback_1..N-1 丢包率彼此完全相同、长间隔对齐——不像带宽竞争的随机丢包。再用「1:20 类计数不随 N 增长」坐实
2	dead-man 看门狗竞态	采集窗口 90 s 与 GRACE 90 s 撞在一起，qdisc 在窗口末尾被自动拆除，N=16 那一轮测的是没有 QoS 的裸链路	自查判据：根类 1:1 的字节增量应 ≈ netdev tx 增量。N=1~8 比值 1.001~1.022（在位），N=16 是 -0.141（中途被拆）
3	设备侧进程清理全程空操作	清理调用写的是 pkill，而设备是 BusyBox 没有 pkill（返回 127），又被 true 吞掉。 后果： 整个 PoC 期间清理从未生效，累积 24~27 个 tcpdump 残留进程	手工核查设备进程时发现残留数量异常
4	rollback.sh 在错误的机器上执行	自称「RG660MK 侧回滚」，但每条命令都没有 SSH 包装，全在 PC 本机跑（本机没有 br-lan）；且每条都带 echo，失败被翻译成友好文案，最终打印「回滚完成」并返回 0	核查「测试结束清理 tc 与临时进程」这条约束时发现它没有任何可用实现
5	pgrep -f 自匹配	ssh host 'pgrep -f camview' 会返回自身 PID（那条 ash 的 cmdline 就含该字符串）。 后果： 校验类判据恒真——「服务存活」永远报运行中，camview 真崩了会被掩盖	修复缺陷 4 时顺带验证，发现校验本身不可信
6	设备侧脚本没有部署步骤	设备侧脚本靠手工 scp，仓库里没有任何同步或校验。PC 侧改了端口段而设备侧还是旧版本时不会报错，只会静默测出错误结论；/tmp 是 tmpfs，重启即清空	修复缺陷 3 做故障注入验证时，误删了设备上的脚本，暴露出复现路径本就不可靠

修复过程中差点再踩中的三个坑（值得记录）

1. 把 `pkill -f X` 换成 `killall -f X` —— **BusyBox 的 killall 不支持 -f**，它把 `-f` 当信号名解析，同样什么也不杀；
2. 按进程名 `killall python3` —— 设备上常驻 **7 个生产 python 进程**（`camav / camview6 / voice_assistant / ai_service / hermes_gateway / mqtt_bridge / cap_watch`），等于直接把产品干掉；
3. `killall sh` —— 会误伤 `/bin/sh /etc/restart_usb.sh` 这类 shell 常驻服务。

最终实现：清理逻辑收敛为单一模块 `rtg_device_cleanup.py`（此前同一个错误被复制了 4 份），tcpdump 按进程名杀，测试脚本一律 `pgrep -f` 取 PID 后**精确 kill**，看门狗靠删除 arm 文件令其自行退出。**验证方式**：故障注入 —— 伪造一个泄漏进程，确认能检出、能杀掉、7 个生产进程全部无恙、`ps` 独立交叉验证残留为 0。

3.3 建立了可复现、可自查的测试框架

- **一键部署**：`deploy_device_scripts.sh` 从 `scripts/` 单向同步设备侧脚本，并**逐文件比对 md5**，不一致即非零退出 —— 杜绝两端漂移；
- **分类命中硬门禁**：每轮自动校验「1:20 类计数增量 vs 接收端实测流量」，比值 < 0.5 即判定该轮无效。判据基于「收到的字节不可能多于离开该类的字节」，是证明式的，不依赖阈值调参；
- **qdisc 在位自查**：用「根类计数增量 $\approx$ netdev tx 增量」识破静默失效，不依赖 qdisc 自身是否存在；
- **判据与取数分离**：`rejudge_scale.py` 可在不重测的前提下用新阈值重新判定已归档数据 —— 避免「改判据就要重跑」，从而避免为迎合结论而调参。

副产品：修复过程中还建立了「近零基线上不能用比值判据」的方法学结论 —— $N=1$ 的抖动基线只有 0.5 ms，用「劣化超过 3 倍」做门禁会把 1.3 ms 的绝对变化判成失败。改为与媒体需求挂钩的绝对预算（ $P95 \leq 5\text{ ms}$ 、 $P99 \leq 10\text{ ms}$ ，即 20 ms 帧间隔的 25%/50%）。

3.4 定量给出了设备的能力边界与瓶颈机理

不只是给出「最大 $N=2$ 」这个数字，而是给出了**为什么**：FEEDBACK 每路 134 kbit/s，1:20 类的 `rate` 决定抖动阈（ ≈ 1.9 路）、`ceil` 决定丢包阈（ ≈ 7 路），两道门槛先后失效，且与类别计数饱和值吻合。有了这个模型，调参就变成了算术题而不是试错（见第五章）。

3.5 发现并纠正了业务口径偏差

补充说明 § 五 假设麦克风为 **16 kHz** / 20 ms 帧，设备侧实测为 **8 kHz** / 20 ms / 320 B。这直接影响带宽预算的计算（若按 16 kHz 估算会高估一倍）。已按实测值修正，并建议产品侧确认正式形态下的采样率。

四、遗留了什么

4.1 【最关键】CONTROL 未被纳入 QoS，核心目标未证明

这是本 PoC 最重要的遗留项

三业务**不共享出口**：VIDEO / FEEDBACK 走 `br-lan` 到 PC，而 CONTROL 走 `ccmni2`（5G）到 `open.feishu.cn`。QoS 只挂在 `br-lan` 上，因此 **CONTROL 的出口队列从头到尾没有被改动过**。

硬证据：所有 QoS ON 轮次的 1:10 类计数 `cls_control_bytes` 均为 0。分类器已下发、规则经 `tc filter show` 核对无误——也就是说「没命中」不是配置问题，而是**流量物理上就不经过这条队列**。

因此 2.2 表中 CONTROL-NET 那 12.4% 的「改善」是公网抖动，不是 QoS 的功劳（OFF/ON 区间重叠即为证）。本 PoC 能证明的是 **br-lan 侧三流分类调度的有效性，不能证明 CONTROL 端到端的 QoS 有效性**。

按设计文档 § 6.1 要求，此处**记录为局限**，未通过修改默认路由去「统一出口」——那既违反「不永久修改网络拓扑」的硬约束，也会让测得的数不再是这台设备真实部署下的数。

4.2 CONTROL-APP（端到端事务时延）完全未采集

设计文档 § 三 要求 CONTROL 指标包含飞书 → Hermes → 应答的端到端事务（`control_transactions.csv`）。本次**未采集**，`control_transactions.csv` 仅有表头。

原因：自动收发需要飞书自建应用的 `FEISHU_APP_ID` / `FEISHU_APP_SECRET`，该凭据不在 Hermes 配置文件中。从**运行中的 gateway 进程提取凭据直接违反**「不泄露 token / secret / cookie」的硬约束，因此没有执行，也没有尝试绕过。

直接后果：设计文档 § 八 中 4 条 CONTROL 判据（P95 改善 $\geq 30\%$ 、P99 改善 $\geq 30\%$ 、时延波动改善 $\geq 30\%$ 、超时率下降）**全部为「未测」**，无法给出结论。

4.3 一拖多能力偏低（N=2）

瓶颈已定位为 1:20 类的参数配置，**不是设备算力或链路带宽的硬限制**——CPU 在 N=16 时也仅 26.7%。但当前参数下 N=2 显然不满足实际业务需要。

4.4 缺少 QCS8550 的对照实测

本 PoC 没有取得 QCS8550 的任何实测数据，因此只能给结构性判断，不给数字对比。设计文档 § 15.3 第 23 问要求的「最大技术差距」目前只能定性回答。

4.5 其他遗留

项	说明
CPU 数字为上界	因缺陷 3，测量期间设备上有残留 tcpdump 进程，会抬高读数。实测 15.9%~26.7% 距 85% 阈值很远，结论不受影响，但绝对值应在清理后重测一次
CONTROL 出口拥塞未能构造	ccmni2 空闲 TX 仅约 45 kbit/s，6 Mbit 瓶颈压不出真实拥塞，却会人为限制生产 CONTROL/LLM 路径，故未对其下发 qdisc
VIDEO 仅 2 fps	现场 camview 既有配置，未修改（遵守「不修改 camview 持久配置」）
无 PTP	设计文档 § 4 明确排除，因此不做单向时延，jitter 只用接收端单一 monotonic clock
无效数据已隔离保留	分类器缺陷期间产生的数据保留在 <code>scale_invalid_classifier_bug/</code> ，未混入统计

五、预计如何解决

5.1 CONTROL 纳入 QoS（对应 4.1）

三个可选路径，按推荐度排序：

序	方案	做法与代价	前置条件
A 推荐	在正式网关（QCS8550）上按设计文档部署 双口透明网关 ，使 CONTROL 与媒体在同一出口排队	最贴近目标形态，一次到位；本套脚本可直接复跑，只需替换设备地址	8550 硬件到位
B	对 <code>ccmni2</code> 也下发 QoS，使 CONTROL 与 WAN 侧流量在同一队列竞争	改动最小， 能立即验证 CONTROL 的 QoS 有效性 。但 <code>ccmni2</code> 是生产上行，人为加瓶颈有拖慢真实业务的风险，且 LTE 实际速率波动大，"6 Mbit" 未必是恰当的人工瓶颈	需人工确认 ，且必须在维护窗口内短时执行（dead-man 看门狗已就绪）
C	策略路由把飞书 IP 导向 <code>br-lan</code> ，实现物理统一出口	能构造出真实的三流竞争，但需要 PC 侧具备转发/NAT 能力	违反「不永久修改网络拓扑」， 需人工确认

建议：若近期就能拿到 8550，走 A；若需要在此之前先拿到 CONTROL 的 QoS 数据，用 B 在维护窗口做一轮短测（脚本与回滚机制已就绪，风险可控）。

5.2 补齐 CONTROL-APP 采集（对应 4.2）

- 需要授权：**飞书自建应用凭据（`FEISHU_APP_ID` / `FEISHU_APP_SECRET`）及一个专用测试群；
- 凭据通过环境变量注入，**不写入任何文件、不进入报告**；
- 执行 `feishu_control_probe.py`（脚本已就绪），采集 `control_transactions.csv` 全部字段（`seq` / `send_ts` / `hermes_rx_ts` / `reply_ts` / `ack_ts` / `rtt_ms` / `success` / `timeout`）；
- 按补充说明 § 九：若样本数 < 100，**不把 P99 作为正式验收结论**，重点用 P50 / P95 / max，P99 只标注为「观察值」。**不为凑 P99 而暴力提高飞书发送频率**；
- 同时补算 `control_latency_variation(i) = |rtt(i) - rtt(i-1)|`，报告中命名为「**CONTROL 应用事务时延波动**」，不冒充纯网络 jitter。

注意

该步骤会**外发真实飞书消息**，务必使用专用测试群，不要打到生产群。

5.3 提升一拖多能力（对应 4.3）

瓶颈是 1:20 类的参数，可按实测码率反算，无需试错：

目标 N	单路码率	建议 rate（抖动阈）	建议 ceil（丢包阈）	约束检查
N = 4	134 kbit/s	≥ 800 kbit/s	≥ 1.1 Mbit/s	保底和 6080 k > 6000 k，需把 VIDEO 保底从 4 M 降到 3.5 M
N = 8	134 kbit/s	≥ 1.6 Mbit/s	≥ 2.2 Mbit/s	保底和将达 6880 k，需重新分配总预算或提高瓶颈
N = 16	134 kbit/s	≥ 3.2 Mbit/s	≥ 4.3 Mbit/s	6 Mbit 瓶颈下不可行，须提升链路或降低单路码率

更根本的两个方向：

- 降低单路码率：当前 8 kHz/16 bit/20 ms 已是较低配置，若正式形态采用音频压缩（如 Opus），单路需求可降到 20~30 kbit/s，N 的上限会显著提高；
- 确认真实链路预算：本次 6 Mbit/s 是为构造拥塞而人为设定的瓶颈。若正式部署的上行远大于此，一拖多的实际能力会高得多——建议先在目标场景实测上行带宽，再定 QoS 参数。

调整后必须重跑验证：`--levels 1,2,4,8,16`，并检查分类命中门禁与根类计数自查是否通过。

5.4 补齐 QCS8550 对照（对应 4.4）

本套脚本已做到「设备无关」：设备地址、接口名、PC IP 均为变量或参数。拿到 8550 后只需：

- 确认目标接口名（当前硬编码 `br-lan`，需参数化）；
- 执行 `deploy_device_scripts.sh` 同步脚本并校验 md5；
- 复跑 `run_ab_test.py` 与 `run_scale_test.py`，用同一套阈值判定；
- 对比 `comparison.csv` 与 `scale_summary.csv`。

预计工作量：半天以内（不含硬件准备）。

5.5 收尾项

项	动作
CPU 数字重测	清理机制已修复，重跑一轮即可得到不含脚手架开销的干净数字
无效数据	<code>scale_invalid_classifier_bug/</code> 已隔离，建议随报告归档保留以备追溯，不删除
麦克风采样率口径	建议产品侧确认正式形态的采样率与编码方式（实测 8 kHz，文档假设 16 kHz）
未分类流量评估	未被显式分类的流量（ICMP 已归类，DNS/NTP 等未归类）会落入 1:40，拥塞时被尾丢。正式部署需逐项确认哪些协议要显式归类
持久化 QoS	本 PoC 为运行时 <code>tc</code> （遵守「不写 <code>/etc/config</code> 」）。正式部署需设计开机自动下发与失败回退机制

附录 A 交付物清单

路径	内容
scripts/	29 个脚本：勘察、QoS 下发/状态/回滚、A/B 编排、一拖多、聚合、出报告、设备部署
logs/control_transactions.csv	CONTROL-APP 事务明细（仅表头，见 4.2）
logs/net_probe_ping.log	NET_PROBE 原始逐包记录
baseline/	A/B/C 全部轮次原始数据 + comparison.csv + improvement.json
qos/	实际下发的 qdisc / class / filter 的 tc dump 与状态快照（取证用）
scale/	一拖多各级原始数据 + scale_summary.csv + 逐级 verdict.json
scale_invalid_classifier_bug/	分类器缺陷期间的数据，已隔离，未进入统计
report/RG660MK_RTG_PoC_测试报告.md	完整测试报告（本文件为其部署视角摘要的完整版）
report/00_inventory.md • 01_stream_check.md	设备台账与业务流勘察
report/comparison.csv • scale_summary.csv	两项测试的结果汇总表

附录 B 复现步骤

```
sh scripts/deploy_device_scripts.sh      # 第 0 步：同步设备侧脚本并校验 md5
sh scripts/discover.sh                   # Phase A 只读勘察
sh scripts/hermes_feishu_discover.sh     # CONTROL 五元组定位与隔离核验
python3 scripts/run_ab_test.py \         # Phase C/D/E: A/B/C 各 3 轮
    --cases idle,idle_qos,congested_qos_off,congested_qos_on \
    --rounds 3 --measure 90 --with-control --outdir baseline
python3 scripts/aggregate.py --dir baseline \
    --out-csv baseline/comparison.csv \
    --improvement-json baseline/improvement.json
python3 scripts/run_scale_test.py \       # Phase F: 一拖多
    --levels 1,2,4,8,16 --measure 90 --with-control --outdir scale
python3 scripts/make_report.py            # 生成测试报告
sh scripts/rollback.sh                   # Phase G: 回滚（复核不通过则非零退出）
```

附录 C 安全约束遵守情况

约束	遵守情况
不刷机	未刷机
不重启 RG660MK	uptime 持续增长，未重启
不覆盖未知现有 QoS	下发前判定既有 qdisc；实测下发前为内核默认
不修改 camview 持久配置	只读其 HTTP 端点，未改任何配置文件
不泄露 token / secret / cookie	未提取任何凭据；抓包只取五元组与长度
不把所有 TCP/443 标记成 P0	按目的 IP 精确匹配飞书，LLM 流量留在 BEST_EFFORT
测试结束清理 tc 与临时进程	最终已清，但过程中一度失效（见 3.2 缺陷 3/4）。修复后实跑 <code>rollback.sh</code> 退出码 0，并经故障注入验证
异常立即中止并回滚	dead-man 看门狗：keepalive 超时 90 s 自动拆除 qdisc。该机制曾误触发一次（见 3.2 缺陷 2），属调用方喂 keepalive 的方式有缺陷，看门狗本身按预期工作

如需完整技术细节

本报告为部署视角的完整版摘要。全部原始数据、逐轮明细、缺陷定位过程与自查判据，见 [report/RG660MK_RTG_PoC_测试报告.md](#)。